计算机操作系统实验报告

实验4.1：两个进程相互通信

姓名：王长儒 学号：2412410023

依据：《计算机操作系统实验指导》第4章第4.3节，正文第48—53页。

一、实验目的与任务

理解进程通信的概念，掌握Linux IPC及套接字通信方法。实现裁判和两名选手的石头、剪刀、布游戏，支持多轮比赛、限时出招、结果反馈与记录，并将程序扩展为网络运行方式。

二、实验环境

【运行后填写①】实验日期：_____；Linux发行版：_____；内核版本：_____；GCC版本：_____。

采集方法：依次执行 date +%F、cat /etc/os-release、uname -r、gcc --version，记录实际使用的环境。

三、数据结构与判定规则

约定0为石头、1为剪刀、2为布。a=b时平局；(a+1)%3=b时A胜，否则B胜。结果编码为0平局、1为A胜、2为B胜、3为双方无有效出招。九种合法组合的预期结果为：00平、01 A胜、02 B胜；10 B胜、11平、12 A胜；20 A胜、21 B胜、22平。

每条报文包含4个32位整数：轮次、A出招、B出招、结果。使用网络字节序统一编码。请求报文只使用轮次字段，选手回复使用轮次与第二字段，裁判反馈包含全部字段。轮次0表示结束。

四、进程通信与网络设计

本地模式使用两组AF_UNIX socketpair。父进程作为裁判，fork产生两个常驻选手进程，每名选手只保留自己的端点。套接字属于Linux IPC机制，指导书的System V消息队列实现仅是示例，因此本实现使用合法的另一种IPC方法。

网络模式将端点替换为TCP连接。服务端依连接顺序指定A、B，两个客户端可位于不同主机。游戏协议和判定逻辑与本地模式共用。服务端监听所有IPv4接口，客户端填写服务端实际地址。TCP为字节流，程序循环收发完整16字节报文，处理短读、短写与EINTR。

裁判每轮设置统一的单调时钟截止时间，两个选手共用2秒出招窗口，避免顺序接收给第二人额外时间。缺失、非法或轮次不符的出招视为无效；一方无效判负，双方无效单独统计。异常轮记录并反馈后结束比赛，防止迟到报文污染下一轮。正常情况下执行指定轮数。结果通知另设发送时限。

五、编译和实验步骤

将附件game.c置于Linux目录，执行以下命令。本地运行会创建三个进程，比赛结束后waitpid回收子进程，关闭套接字并保存结果文件。

```
gcc -std=c11 -Wall -Wextra -O2 game.c -o game
./game local 100
cat result.csv
```

网络测试依次在三个终端执行；跨机器时把127.0.0.1改为服务端IPv4地址，并确保端口可达。服务端等待两个连接后开始计时。

```
./game server 100 9000
./game client 127.0.0.1 9000
./game client 127.0.0.1 9000
```

六、结果记录与分析方法

预期每轮打印轮次、双方出招和判定，并将同样内容写入result.csv。客户端收到双方出招与结果，终端显示player行。正常100轮应有100条数据记录，平局数+A胜数+B胜数=100；最终胜者取胜局数较多的一方，胜局相等为总平局。源程序已输出总体胜者，统计数输出至终端。实际结果填写在后附的运行记录页中。

建议验证：①运行1轮、10轮和100轮，逐行按九种组合核对；②暂停一个客户端超过2秒，确认另一方获胜且比赛结束；③中断一个客户端，确认服务端不永久等待；④分别在本地和两台主机上运行，确认反馈与记录一致。双方无效时不应强行计为平局。

七、实验思考

1. 随机数如何合理取值？fork会复制伪随机数状态，若两个子进程使用同一秒的时间种子，容易产生相同序列。程序把时间与不同PID结合，每个进程只初始化一次，并用rand_r保存独立状态。rand_r()%3仍有轻微取模偏差，且不适用于安全场景；更严格实现可使用系统随机源并拒绝采样，固定种子则适合复现实验。

2. IPC机制如何选择？匿名管道适合亲缘进程单向字节流，双向需两条管道；FIFO适合本机无亲缘进程；消息队列保存消息边界并可按类型接收；共享内存适合大数据低复制通信，但需信号量等同步；信号量主要用于同步与互斥；UNIX域套接字适合本机双向通信，TCP套接字适合跨主机可靠字节流通信。

八、问题修正与总结

指导书示例的消息结构应把long类型消息类型放在首部，msgsnd/msgrcv长度应仅计算消息正文；示例胜负公式不能完整覆盖循环胜负关系，且等待子进程后阻塞接收不能满足限时出招要求。本实现改用明确的模运算、固定报文和截止时间。通过共用协议，本地IPC和网络通信可复用游戏逻辑；实验通过轮次字段关联请求与响应，以截止时间约束出招，以文件保存逐轮结果。运行结论根据后附实测记录填写。

局限：网络接入等待尚未设超时；客户端连接亦使用系统默认超时。随机策略不保证胜率恰好各三分之一，不能把统计接近均匀当作每次运行的必然结果。

运行记录：本地IPC与网络通信

红色【运行后填写】为需要补充的内容，其余正文已完成。请用实际输出填写，不必保证双方胜率相等。每次运行会覆盖result.csv，须分别备份。

【运行后填写②】编译与本地100轮结果

执行：gcc -std=c11 -Wall -Wextra -O2 game.c -o game；随后执行 ./game local 100 > local.log 2>&1；cp result.csv local_result.csv。

编译是否成功：_____；警告或错误及处理：_____。

实际完成轮数：____；A胜：____；B胜：____；平局：____；双方无效：____；总胜者：____。正常100轮检查：A胜+B胜+平局是否等于100：____。

从local_result.csv摘录前3轮（轮次、A、B、结果）：

第1轮：_____；第2轮：_____；第3轮：_____。

截图①：插入编译命令与本地运行末尾统计的终端截图。

【运行后填写③】网络100轮结果

三个终端分别执行正文网络命令。跨主机实验填写服务端实际IP；同机127.0.0.1可验证TCP逻辑，但不作为跨主机验证。结束后执行 cp result.csv network_result.csv。

测试方式（同机/跨主机）：____；服务端IP：_____；客户端A/B所在主机：_____。

实际完成轮数：____；A胜：____；B胜：____；平局：____；双方无效：____；总胜者：____。

两客户端是否收到双方出招与结果：____；CSV与服务端输出是否一致：____。

截图②：插入服务端统计及两个客户端收到结果的截图。

运行记录：超时、断连与最终结论

【运行后填写④】出招超时测试

为便于操作，先启动服务端和客户端A，在客户端B连接前执行 kill -STOP
客户端A的PID，再连接客户端B。等待裁判超时判定后，执行 kill -CONT 客户端A的PID
恢复它。仅暂停本次游戏的客户端进程。

记录：超时轮次____；A出招字段____；B出招字段____；结果码____；程序是否结束____。预期A无有效出招，若B有效则结果码为2。

实际现象及分析：_____。

截图③：插入暂停命令与服务端超时判定截图。

【运行后填写⑤】客户端断开测试

先连接客户端A，再在其终端按Ctrl+C，然后连接客户端B。记录服务端是否识别连接关闭并结束比赛。

实际结果：_____。

截图④：插入服务端断连后的输出。

【运行后填写⑥】实测结论

本地IPC测试：____；TCP通信测试：____；超时测试：____；断连测试：____（分别填写通过或未通过）。

根据实测结果，程序_____。

若存在异常，记录触发条件、错误信息及修正情况：_____。

填写提示：只有实际通过的项目才填写“通过”；若仅做了同机TCP测试，应注明尚未完成跨主机验证。截图按①至④顺序附于报告末尾，正文不预留大块空白。

附录：完整C源程序（同时见game.c）

```
#define _POSIX_C_SOURCE 200809L
#include <sys/socket.h>
#include <sys/wait.h>
#include <arpa/inet.h>
#include <poll.h>
#include <unistd.h>
#include <signal.h>
#include <time.h>
#include <stdio.h>
```

```
#include <stdlib.h>
#include <stdint.h>
#include <errno.h>
#include <string.h>
/* Fixed-size, network-byte-order frames: round, moveA, moveB, result. */
static void die(const char*s){perror(s);exit(1);}
static long long now(void){struct timespec t;clock_gettime(CLOCK_MONOTONIC,&t);return
t.tv_sec*1000LL+t.tv_nsec/1000000;}
static int transfer(int fd,uint32_t*v,int sending,long long end){
 size_t n=0;unsigned char*b=(unsigned char*)v;
 while(n<16){int left=(int)(end-now());if(left<=0)return -1;
 struct pollfd p={fd,sending?POLLOUT:POLLIN,0};
 int r=poll(&p,1,left);if(r<0&&errno==EINTR)continue;if(r<=0)return -1;
 ssize_t k=sending?send(fd,b+n,16-n,MSG_DONTWAIT|MSG_NOSIGNAL):recv(fd,b+n,16-n,MSG_DONTWAIT);
 if(k<0&&(errno==EINTR||errno==EAGAIN))continue;if(k<=0)return -1;n+=(size_t)k;
 }return 0;
}
static int put(int f,int r,int a,int b,int z,long long e){uint32_t
v[4]={htonl(r),htonl(a),htonl(b),htonl(z)};return transfer(f,v,1,e);}
static int get(int f,int*v,long long e){uint32_t w[4];if(transfer(f,w,0,e))return -1;for(int
i=0;i<4;i++)v[i]=(int32_t)ntohl(w[i]);return 0;}
static void player(int f){
 unsigned seed=(unsigned)time(NULL)^(unsigned)getpid();int v[4];
 while(!get(f,v,now()+60000)&&v[0]>0){int r=v[0];int m=(int)(rand_r(&seed)%3);
 if(put(f,r,m,0,0,now()+2000)||get(f,v,now()+60000))break;
 printf("player %ld round %d: A=%d B=%d result=%d\n",(long)getpid(),r,v[1],v[2],v[3]);
 }close(f);
}
static void judge(int*f,int rounds){
 FILE*out=fopen("result.csv","w");if(!out)die("fopen");
 fprintf(out,"round,A,B,result\n");int count[4]={0};
 for(int r=1;r<=rounds;r++){
 int moves[2]={-1,-1},alive[2]={1,1};long long end=now()+2000;
 for(int i=0;i<2;i++)if(put(f[i],r,0,0,0,end))alive[i]=0;
 for(int i=0;i<2;i++){int
v[4];if(alive[i]&&!get(f[i],v,end)&&v[0]==r&&v[1]>=0&&v[1]<3)moves[i]=v[1];}
 int a=moves[0],b=moves[1];
 /* 0 rock, 1 scissors, 2 paper; result 0 draw,1 A,2 B,3 both invalid */
 int z=a<0?(b<0?3:2):(b<0?1:(a==b?0:((a+1)%3==b?1:2)));
 count[z]++;printf("%d,%d,%d,%d\n",r,a,b,z);fprintf(out,"%d,%d,%d,%d\n",r,a,b,z);
 for(int i=0;i<2;i++)put(f[i],r,a,b,z,now()+2000);
 if(a<0||b<0){puts("timeout/invalid/disconnect: match stopped");break;}
 }
 printf("draw=%d A=%d B=%d both_invalid=%d\n",count[0],count[1],count[2],count[3]);
 printf("overall=%s\n",count[1]==count[2]?"draw":count[1]>count[2]?"A":"B");
 for(int i=0;i<2;i++){put(f[i],0,0,0,0,now()+100);close(f[i]);}fclose(out);
}
int main(int argc,char**argv){
 if(argc<2){fprintf(stderr,"local N | server N PORT | client IPv4 PORT\n");return 1;}
 if(!strcmp(argv[1],"client")&&argc==4){int
f=socket(AF_INET,SOCK_STREAM,0);if(f<0)die("socket");struct sockaddr_in a={0};a.sin_family=AF_
INET;a.sin_port=htons(atoi(argv[3]));if(inet_pton(AF_INET,argv[2],&a.sin_addr)!=1)return
1;if(connect(f,(void*)&a,sizeof a))die("connect");player(f);return 0;}
 int n=argc>2?atoi(argv[2]):0;if(n<1||n>10000)return 1;int f[2];pid_t p[2];
 if(!strcmp(argv[1],"local")){
 int s[2][2];for(int i=0;i<2;i++)if(socketpair(AF_UNIX,SOCK_STREAM,0,s[i]))die("socketpair");
 for(int i=0;i<2;i++){p[i]=fork();if(p[i]<0)die("fork");if(!p[i]){for(int
j=0;j<2;j++){close(s[j][0]);if(j!=i)close(s[j][1]);}player(s[i][1]);exit(0);}f[i]=s[i][0];}
 for(int i=0;i<2;i++)close(s[i][1]);judge(f,n);for(int i=0;i<2;i++)waitpid(p[i],NULL,0);
 }else if(!strcmp(argv[1],"server")&&argc==4){
 int s=socket(AF_INET,SOCK_STREAM,0);if(s<0)die("socket");int
yes=1;setsockopt(s,SOL_SOCKET,SO_REUSEADDR,&yes,sizeof yes);
 struct sockaddr_in a={0};a.sin_family=AF_INET;a.sin_addr.s_addr=htonl(INADDR_ANY);a.sin_port=
htons(atoi(argv[3]));if(bind(s,(void*)&a,sizeof a)||listen(s,2))die("bind/listen");
 puts("waiting for two players");for(int
i=0;i<2;i++){f[i]=accept(s,NULL,NULL);if(f[i]<0)die("accept");}close(s);judge(f,n);
 }else return 1;return 0;
}
```